AI-Powered Quiz Generator

TechVest Academy Day 5 — Application Build · Paul Quintilian · May 2026

Problem & Solution

Educators and students need a faster way to create practice quizzes from existing study material. Manually writing multiple-choice questions from a slide deck is tedious; quiz tools that exist rarely calibrate difficulty or explain why a wrong answer is wrong.

This project is a Streamlit web application that ingests any PowerPoint (`.pptx`) file, asks a large language model to write multiple-choice questions about the slide content, and presents an interactive quiz with instant scoring and AI-generated explanations for wrong answers. Users choose how many questions (5–30) and at which difficulty level (Simple, Medium, Complex).

Architecture

The application is split into three single-purpose Python files plus a deployed Streamlit web UI. This separation makes each piece independently testable and keeps the AI logic, file parsing, and presentation cleanly decoupled.

File	Responsibility	Tech
pptx_parser.py	Extract all text from a .pptx file	python-pptx
ai_generator.py	Call the LLM, parse strict-JSON MCQ response	OpenAI SDK pointed at OpenRouter; model: anthropic/claude-haiku-4-5
app.py	Three-screen Streamlit UI driven by session_state: Upload → Quiz → Results	Streamlit

Scope Decisions

In Scope (delivered)	Out of Scope (per assignment)
.pptx upload and full-deck text extraction	Audio / video file input
AI-generated MCQs with 4 options each	Essay / open-ended questions
Configurable question count (5–30)	User authentication
Simple / Medium / Complex difficulty calibration	Multi-language support
Score plus per-question explanation of wrong answers	
Responsive Streamlit UI with wide layout for projection	

Implementation Highlights

Strict-JSON prompting. The system prompt instructs the model to return *only* a JSON array (no prose, no markdown, no code fences) with an exact schema for each question (question, options[4], correct_index 0–3, explanation). A defensive parser strips ```` ``` ```` ```` fences if the model adds them anyway.

Difficulty calibration. Rather than passing the difficulty word and hoping the model interprets it consistently, the system prompt defines each level behaviorally: Simple = direct recall; Medium = comprehension and

relationships; Complex = synthesis across multiple slides. Test runs confirmed that Complex questions visibly required multi-section reasoning while Simple stuck to surface-level recall.

Provider-agnostic model selection. Because the OpenAI SDK is pointed at OpenRouter, swapping models is a one-line change. During development we cycled through three providers in response to free-tier rate limits and a deprecated endpoint before settling on Anthropic Claude Haiku 4.5 for reliability.

Secrets handling. The OpenRouter API key lives in a local `.env` file (gitignored, never committed) and in Streamlit Cloud's Secrets vault for the deployed environment. The same `python-dotenv` code reads from either source.

Deliverables

Item	Location
Source code (GitHub)	https://github.com/quintilian-bsa/day5-quiz-generator
Live deployed app	https://day5-quiz-generator-vcnmgghui8jrn66nyxxnpa.streamlit.app
Project report (this document)	Project_Report.pdf
5-minute video demo	https://youtu.be/S-yDgB9ZBac

Lessons Learned

1. Prompt engineering is the heart of LLM apps. The quality of the generated quiz depends almost entirely on how carefully the system prompt is shaped — naming the failure modes ("no prose, no markdown, no code fences"), providing an exact JSON schema, and calibrating difficulty behaviorally. Tightening the prompt produced bigger quality gains than swapping models.

2. Free LLM tiers are unreliable for live demos. OpenRouter's free models hit rate limits and endpoint-deprecation errors during development. Switching to Claude Haiku 4.5 (≈ $0.003 per quiz) eliminated those failures at trivial cost.

3. Three small files beat one big file. Splitting parsing, AI calling, and UI into separate modules made each piece testable in isolation. When the deployed app misbehaved, the failure isolation was instant.

4. Treat secrets defensively. A `.gitignore` entry plus a single-source-of-truth `.env` file is the simplest possible guard against the most common open-source mistake (a leaked API key in a public repo).

Built as part of the TechVest Academy Generative AI course, Module 1 (May 2026). Stack: Python 3.11, Streamlit, python-pptx, OpenAI SDK, python-dotenv, OpenRouter, Anthropic Claude Haiku 4.5.